NTNU

Kunnskap for en bedre verden

TTK4190 - Guidance, Navigation and Control of Marine Craft, Aircraft and Drones

Assignment 2: Part 3

Authors:
Audun Bagøien Hustad
Bendik Letting Skari
Fredrik Siem Taklo
William Hammer

12th November 2025

Table of Contents

1 LOS Guidance Law for Straight-Line Path Following

1a Guidance selection

We choose to use guideance law xx.

Choose delta_h as 600m and R_switch as 500m as we found this to be a good balance between going close enough the way point, and starting to adapt the course for the next point early. These numbers are of course very depended on the mission and the density of the points.

```
% --- LOS params ---
Delta_h = 600; % Lookahead distance
 Rsw = 500; % Switch radius

function chi_d = LOS_guidance(e_y, pi_p, Delta_h)

chi_d = pi_p - atan(e_y / max(Delta_h,1e-6));
chi_d = ssa(chi_d);

end
```

1b Guidance code

```
% --- LOS params ---
Rstop = 100; % End radius
```

and in the main loop:

```
[xk1,yk1,xk,yk,last] = WP_selector(x(4),x(5), WP, Rsw, Rstop);
[e_y,pi_p] = crossTrackError(xk1,yk1,xk,yk,x(4),x(5));
chi_d = LOS_guidance(e_y,pi_p, Delta_h);
psi_ref = chi_d
```

1c Guidance path

The path looks and works as expected. See Figure 1.

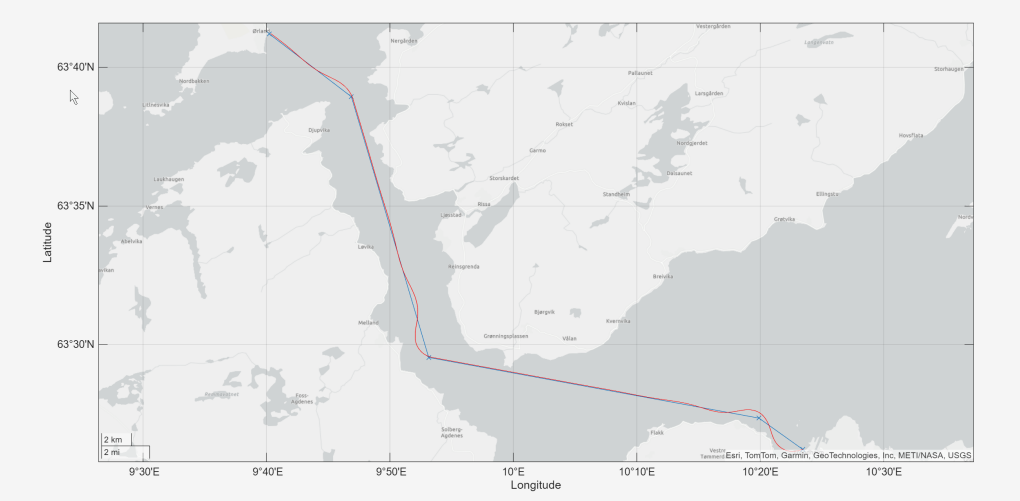

Figure 1: Path of vessel following way points.

2 Task 2

2a 2a

With the current turned off, the β and β_c are identical, meaning that there is no crab angle, thus no need for crab angle compensation.

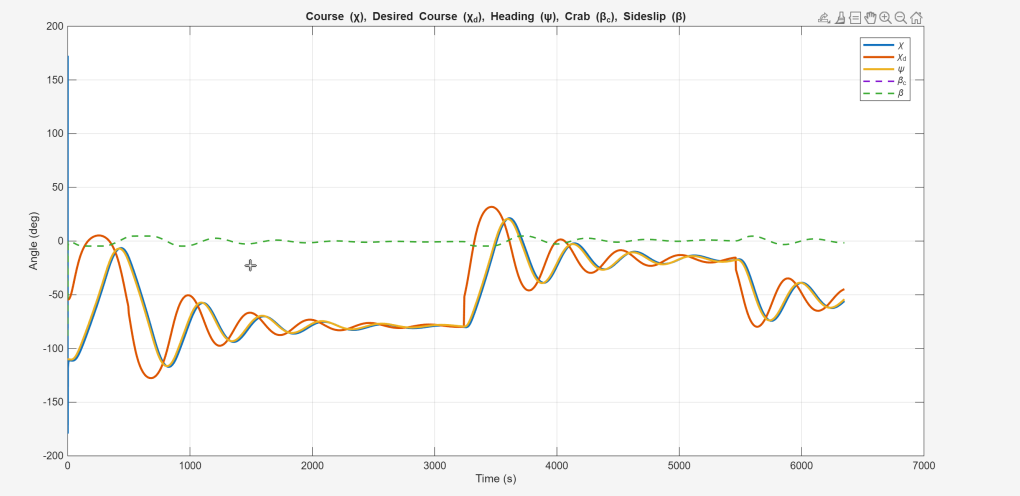

Figure 2: No current

2b 2b

With the current on, and set to 1, we can see that $\beta and \beta_c$ are not identical and we could therefore look into ways to compensate for this.

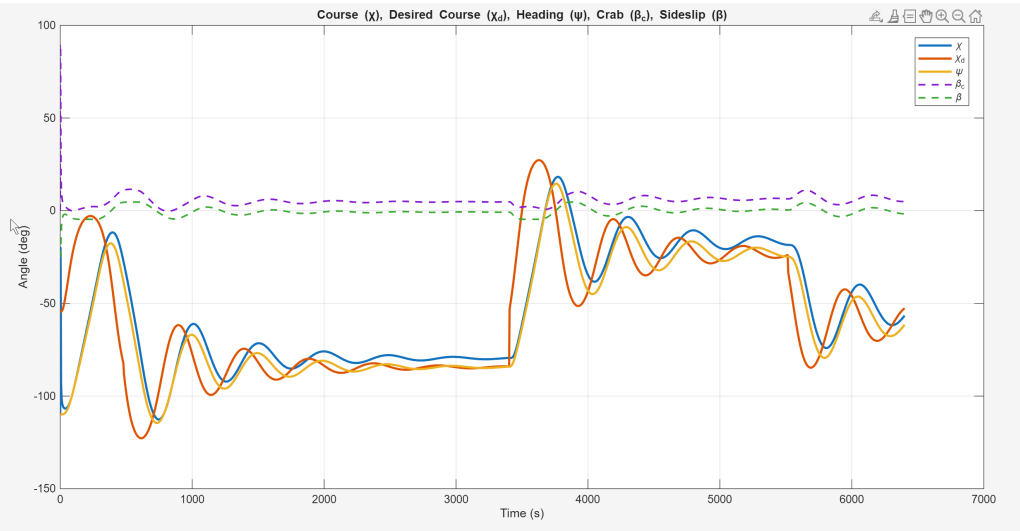

Figure 3: With current

2c 2c

In order to compensate for crab angle, we subtract β_c form psi_ref, so the main block looks like this:

```
[xk1,yk1,xk,yk,last] = WP_selector(x(4),x(5), WP, Rsw, Rstop);
[e_y,pi_p] = crossTrackError(xk1,yk1,xk,yk,x(4),x(5));
chi_d = LOS_guidance(e_y,pi_p, Delta_h);
psi_ref = chi_d - beta_c;
```

The plots over $\chi, \chi_d, \psi, \beta and \beta_c$ are as following:

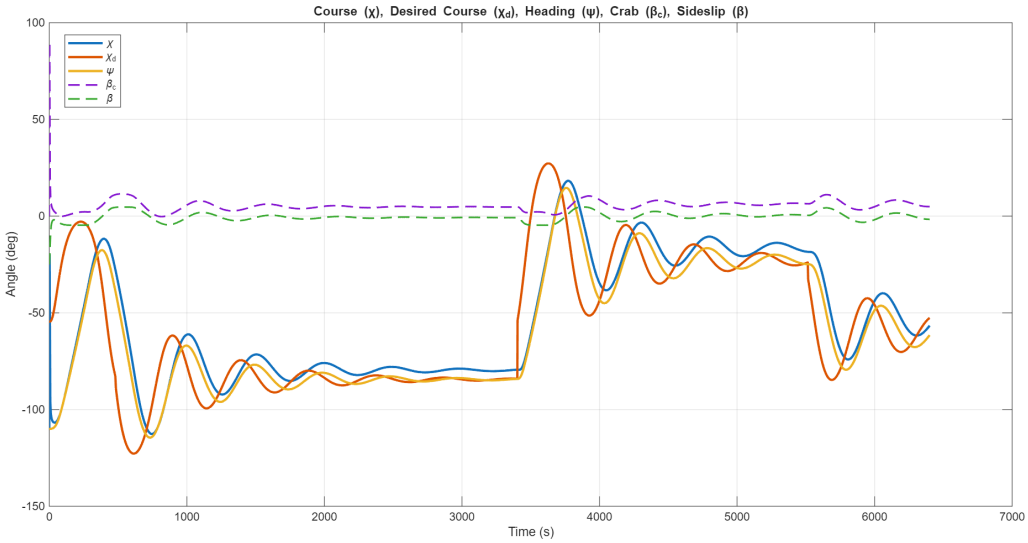

Figure 4: Without crab angle compensation

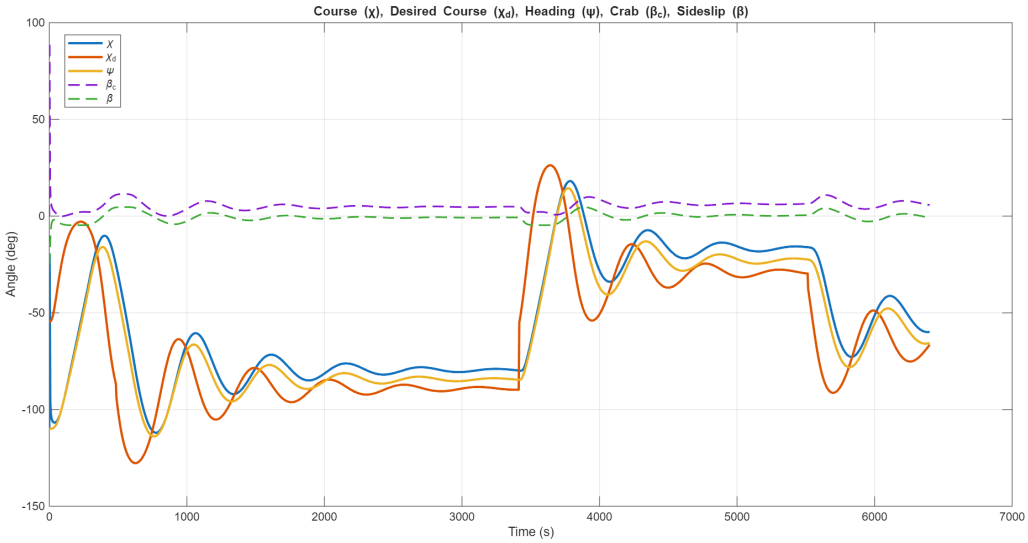

Figure 5: With crab angle compensation

2d 2d

We change our main to:

```
[xk1,yk1,xk,yk,last] = WP_selector(x(4),x(5), WP, Rsw, Rstop);
[e_y,pi_p] = crossTrackError(xk1,yk1,xk,yk,x(4),x(5));
chi_d = LOS_guidance(e_y,pi_p, Delta_h);
psi_ref = ILOS_guidance(e_y, pi_p, kappa, Delta_h, h);
xd_dot = ref_model(xd, psi_ref);
```

with the function as follows:

```
function psi_ref = ILOS_guidance(e_y, pi_h, kappa, Delta_h, h)

% Ensure the integral state is updated correctly
persistent e_y_int;    % integral state

if isempty(e_y_int)
    e_y_int = 0;
end

% ILOS guidance law
Kp = 1 / Delta_h;
Ki = kappa*Kp;
psi_ref = pi_h - atan( Kp * e_y + Ki * e_y_int); % Eq 12.108 in Fossen 2021

% Propagation of states to time k+1
% Eq 12.109 in Fossen 2021
e_y_int = e_y_int + h * Delta_h * e_y/(Delta_h^2 + (e_y + kappa * e_y_int)^2);

end
```

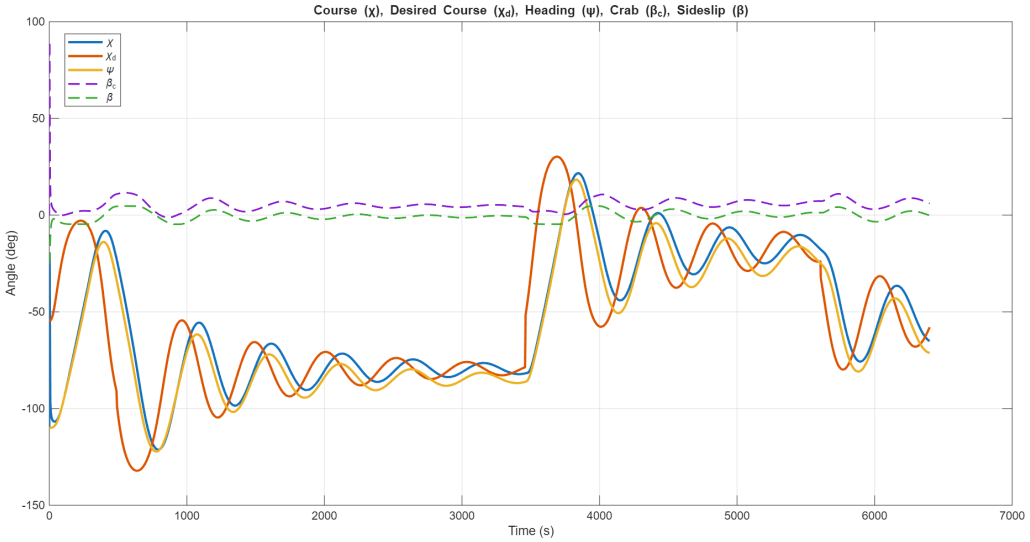

Figure 6: Reaction plot with ILOS and no crab angle compensation

ILOS has the advantage that it will always work, even when the current is unknown, since the integrator automatically compensates for steady-state cross-track errors. But, it responds slower due to the integrator delay. Direct crab angle compensation is faster and more efficient when the current can be measured, but it relies on having accurate current estimates to work properly.

3 Task 3

3a 3a Continuous-time model and Kalman filter matrices

We use a linear yaw (heading) model with states yaw angle ψ, yaw rate r, and a rudder bias b that models a slowly varying disturbance (random walk). The rudder angle δ is the input and the measured output is the yaw angle ψ.

Process model.

$$\dot{\psi}(t) = r(t) + w_\psi(t), \tag{1}$$

$$\dot{r}(t) = -\frac{1}{T}\, r(t) + \frac{K}{T}\big(\delta(t) - b(t)\big) + w_r(t), \tag{2}$$

$$\dot{b}(t) = w_b(t), \tag{3}$$

where $T > 0$ and $K \neq 0$ are the (linear) Nomoto time constant and gain, respectively. The noises $w_\psi(t)$, $w_r(t)$ and $w_b(t)$ are mutually independent zero-mean white processes.

Measurement model.

$$y(t) = \psi(t) + v(t), \tag{4}$$

with $v(t)$ zero-mean white measurement noise.

State-space form. Let $x = \begin{bmatrix}\psi & r & b\end{bmatrix}^\top$. Then

$$\dot{x}(t) = A\, x(t) + B\, \delta(t) + E\, w(t), \tag{5}$$

$$y(t) = C\, x(t) + D\, \delta(t) + v(t), \tag{6}$$

with

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & -\frac{1}{T} & -\frac{K}{T} \\ 0 & 0 & 0 \end{bmatrix}, \qquad B = \begin{bmatrix} 0 \\ \frac{K}{T} \\ 0 \end{bmatrix}, \qquad C = \begin{bmatrix}1 & 0 & 0\end{bmatrix}, \qquad D = \begin{bmatrix}0\end{bmatrix}, \tag{7}$$

$$E = I_3, \qquad w(t) = \begin{bmatrix} w_\psi & w_r & w_b \end{bmatrix}^\top. \tag{8}$$

Noise covariances (spectral densities) are specified as

$$\mathbb{E}\big[w(t)w^\top(\tau)\big] = Q_c\, \delta(t-\tau), \qquad Q_c = \mathrm{diag}(q_\psi,\, q_r,\, q_b), \tag{9}$$

$$\mathbb{E}[v(t)v(\tau)] = R_c\, \delta(t-\tau). \tag{10}$$

Here q_b sets the random-walk intensity of the rudder bias.

3b 3b Discretization (first-order/Euler) and KF implementation matrices

For sampling time $h > 0$, a first-order discretization yields

$$x_{k+1} = F\, x_k + G\, \delta_k + w_k, \tag{11}$$

$$y_k = H\, x_k + v_k, \tag{12}$$

with

$$F \approx I_3 + hA = \begin{bmatrix} 1 & h & 0 \\ 0 & 1 - \frac{h}{T} & -\frac{hK}{T} \\ 0 & 0 & 1 \end{bmatrix}, \tag{13}$$

$$G \approx hB = \begin{bmatrix} 0 \\ \frac{hK}{T} \\ 0 \end{bmatrix}, \qquad H = C = \begin{bmatrix}1 & 0 & 0\end{bmatrix}. \tag{14}$$

If the continuous process noise $w(t)$ has spectral density Q_c, the corresponding discrete process noise covariance is (first-order)

$$Q_d \approx E\, Q_c\, E^\top\, h = Q_c\, h. \tag{15}$$

If R_c is the continuous-time measurement noise spectral density, then the discrete measurement noise variance over a sampling interval is

$$R_d \approx R_c / h. \tag{16}$$

(Alternatively, you can specify R_d directly from the sensor's discrete-time variance.)

Discrete-time Kalman filter recursion. With (F, G, H, Q_d, R_d), the standard DT KF is:

$$\hat{x}_{k|k-1} = F\hat{x}_{k-1|k-1} + G\,\delta_{k-1}, \tag{17}$$

$$P_{k|k-1} = FP_{k-1|k-1}F^\top + Q_d, \tag{18}$$

$$K_k = P_{k|k-1}H^\top\big(HP_{k|k-1}H^\top + R_d\big)^{-1}, \tag{19}$$

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k\big(y_k - H\hat{x}_{k|k-1}\big), \tag{20}$$

$$P_{k|k} = (I - K_kH)P_{k|k-1}. \tag{21}$$

3c 3c Observability

The observability matrix is

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ CA^2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -\frac{1}{T} & -\frac{K}{T} \end{bmatrix}. \tag{22}$$

For $T > 0$ and $K \neq 0$ we have $\operatorname{rank}(\mathcal{O}) = 3$, hence all states ψ, r, and b are observable from yaw measurements. (If $K = 0$, the bias b is not observable.)

4 Task 4

4a Noisy vs true measurements

Code used to generate the noisy measurements.

```
## ---- Before the loop ---- ##
rng(1)                              % reproducibility
sigma_psi = deg2rad(0.5);           % [rad] std of yaw measurement noise
sigma_r   = deg2rad(0.1);           % [rad/s] std of yaw-rate measurement noise

psi_meas_hist = zeros(nTimeSteps,1);
r_meas_hist   = zeros(nTimeSteps,1);
psi_true_hist = zeros(nTimeSteps,1);
r_true_hist   = zeros(nTimeSteps,1);
psi_hat_hist = zeros(nTimeSteps,1);
r_hat_hist   = zeros(nTimeSteps,1);
b_hat_hist   = zeros(nTimeSteps,1);

## ---- In the loop ---- ##
% ----- Noisy measurements (from true states) -----
    psi_true = ssa(x(6));           % x = [u v r x y psi delta n Qm]'
    r_true   = x(3);

    psi_meas = ssa(psi_true + randn*sigma_psi );
    r_meas   = r_true   + randn*sigma_r;   % not used by KF, but useful for plots

    % Log for plotting
    psi_meas_hist(i) = psi_meas;
    r_meas_hist(i)   = r_meas;
    psi_true_hist(i) = psi_true;
    r_true_hist(i)   = r_true;
```

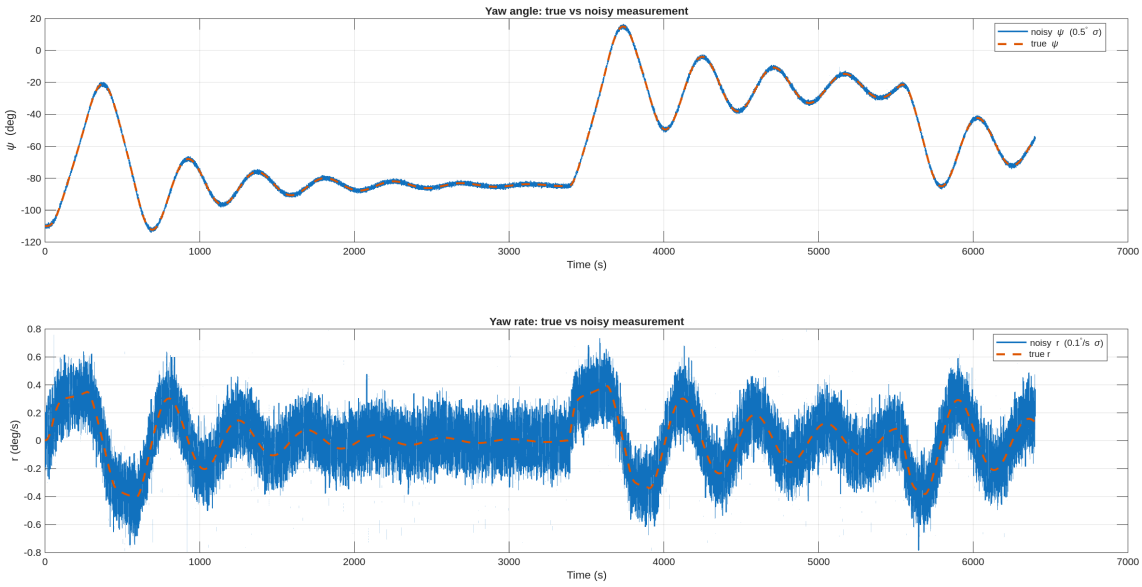

Figure 7: Comparing noisy measurements with true measurements.

4b Implementation

The Kalmanfilter was implemented with this code.

```
function [x_pst,P_pst,x_prd,P_prd] = KF(x_prd,P_prd,Ad,Bd,Ed,Cd,Qd,Rd,psi_meas, delta_c)

% Discrete-time Kalman filter for x = [psi; r; b]
% Inputs:
%   x_prd,P_prd : prior state and covariance
%   Ad,Bd,Ed,Cd : discrete model matrices
%   Qd,Rd       : process/measurement noise covariances
%   psi_meas    : yaw angle measurement (rad)
%   delta       : rudder input (rad)
% Outputs:
%   x_pst,P_pst : posterior state and covariance
%   x_prd,P_prd : predicted state and covariance for next step

% --- Corrector ---
Kk    = P_prd*Cd' / (Cd*P_prd*Cd' + Rd);
innov = ssa( psi_meas - Cd*x_prd );  % wrap angle innovation
x_pst = ssa(x_prd + Kk*innov);
IkC   = eye(size(P_prd)) - Kk*Cd;
P_pst = IkC*P_prd*IkC' + Kk*Rd*Kk';

% --- Predictor ---
u     = delta_c;                          % rudder input to Nomoto model
x_prd = ssa(Ad*x_pst + Bd*u);
P_prd = Ad*P_pst*Ad' + Ed*Qd*Ed';

end
```

These are the following plots of the comparison.

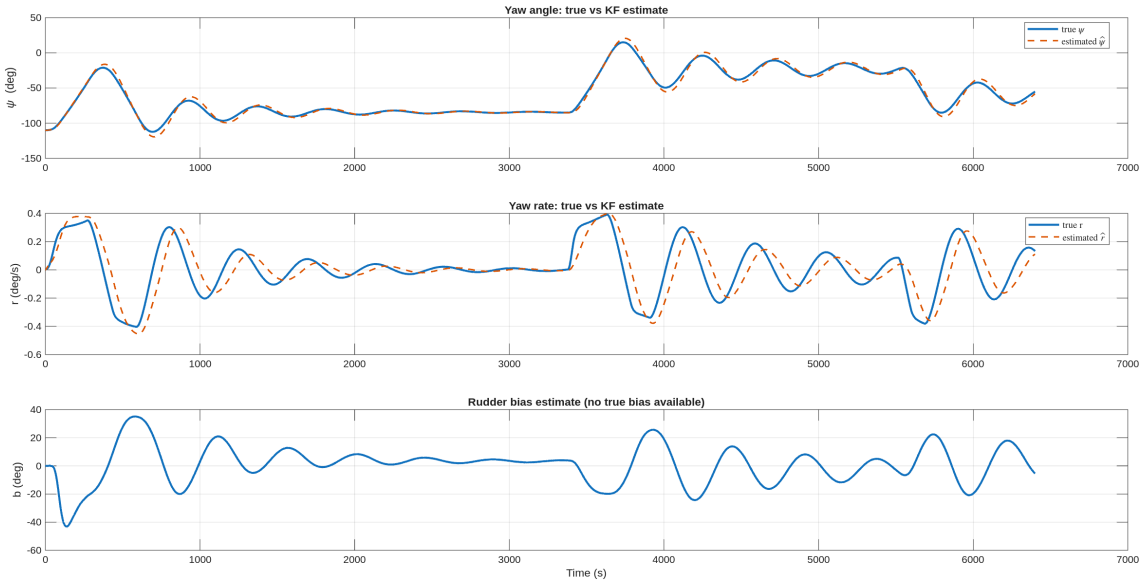

Figure 8: Comparing true states with the estimated ones.

The tuning of the matrices $\mathbf{Q}_d$, $\mathbf{R}_d$, and $\mathbf{P}(0)$ was performed by trial and error by comparing plots of the measured and estimated states, and by testing pure estimation performance. Since the rudder command signal contained significant noise, the process noise covariance $\mathbf{Q}_d$ was chosen to

be very small. This causes the estimated states to lag slightly behind the true values, particularly noticeable for the yaw rate, as shown in Figure 8. The final values used were the following.

$$\mathbf{Q}_d = \begin{bmatrix} 1 \times 10^{-11} & 0 \\ 0 & 1 \times 10^{-7} \end{bmatrix}, \quad \mathbf{R}_d = \left(\frac{0.5^\circ \pi}{180}\right)^2, \quad \mathbf{P}(0) = \text{diag}\left(\left[\left(\frac{30^\circ \pi}{180}\right)^2, \left(\frac{0.01^\circ \pi}{180}\right)^2, \left(\frac{1.0^\circ \pi}{180}\right)^2\right]\right)$$

4c Simulation with noisy measurements, no estimation

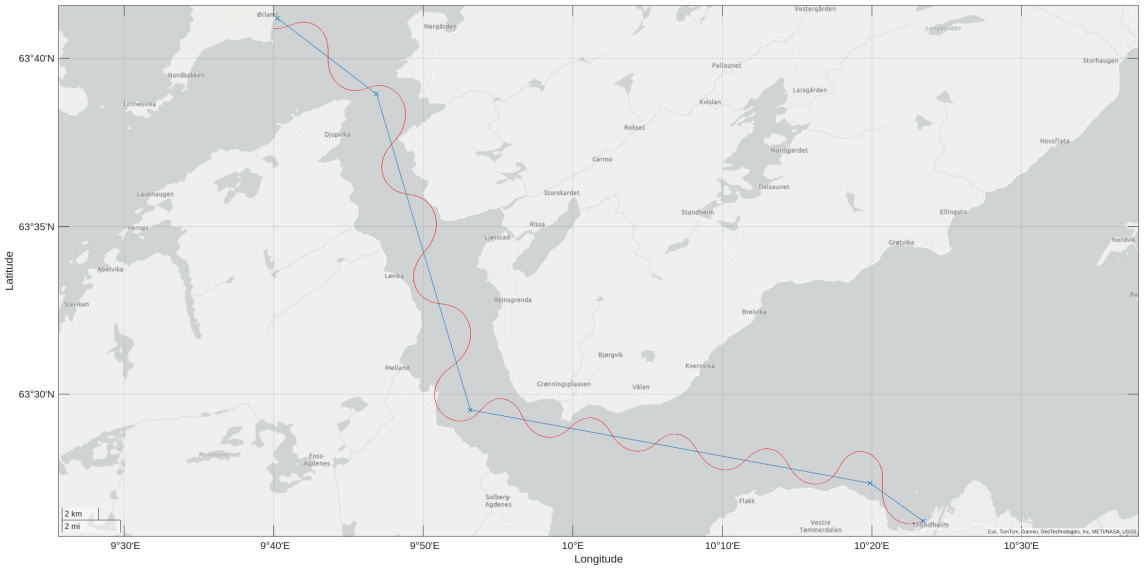

Figure 9: Trajectory of the vessel on map.

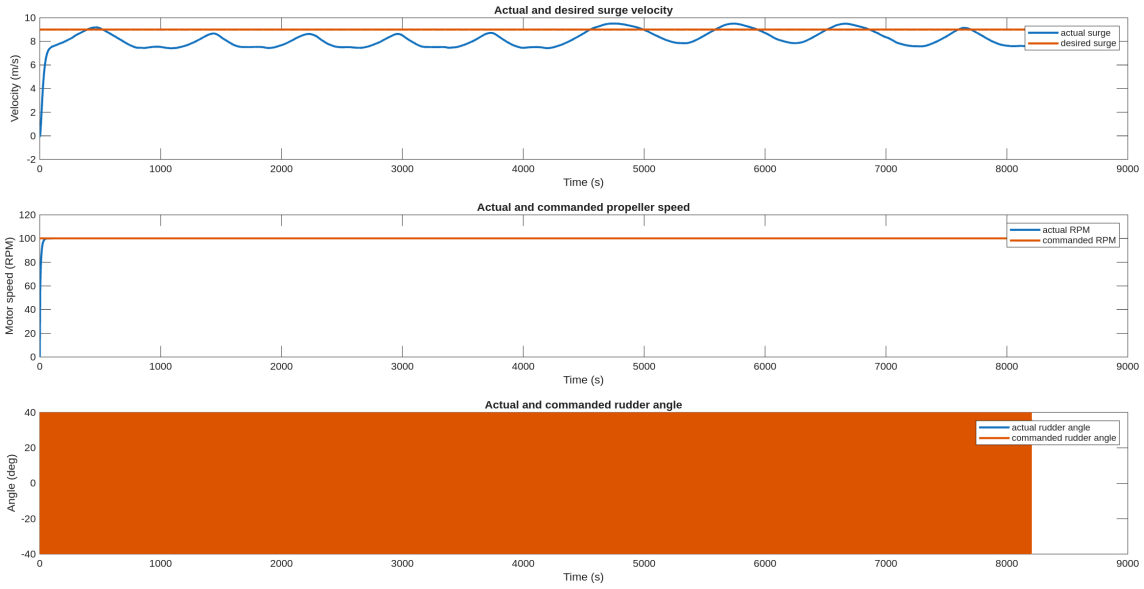

Figure 10: Commanded rudder angle versus actual rudder angle.

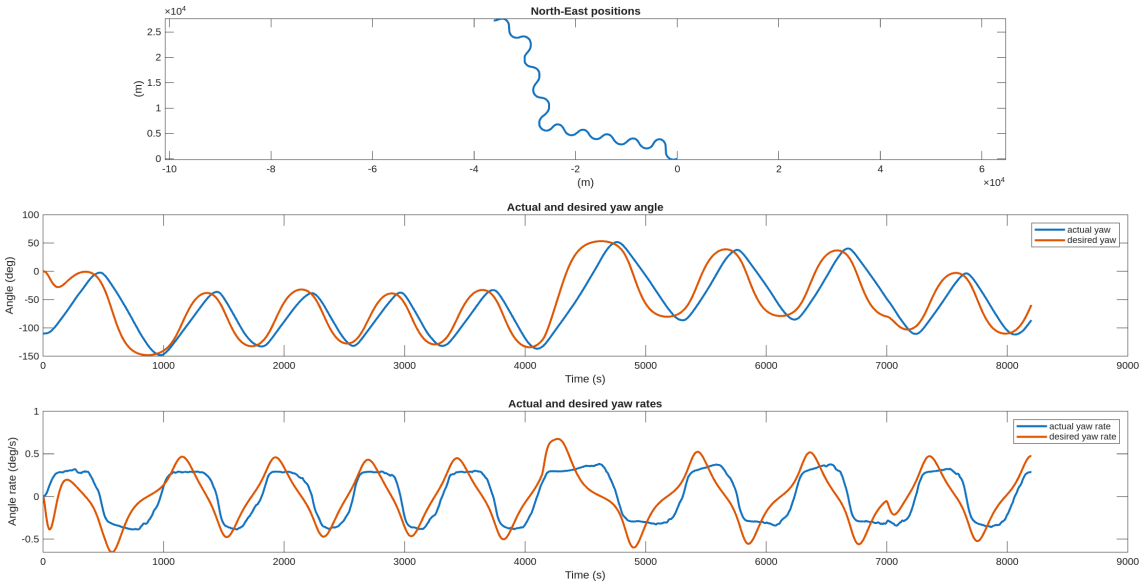

Figure 11: Desired yaw angle and yaw rate versus actual values.

As seen from the figures, the vessel struggles to follow the desired trajectory. The large amount of measurement noise, particularly in the yaw-rate signal, causes the controller to react to each noise spike. This results in large rudder activity and overall unstable motion.

4d Simulation with estimation

Figure 12: Trajectory of the vessel on map.

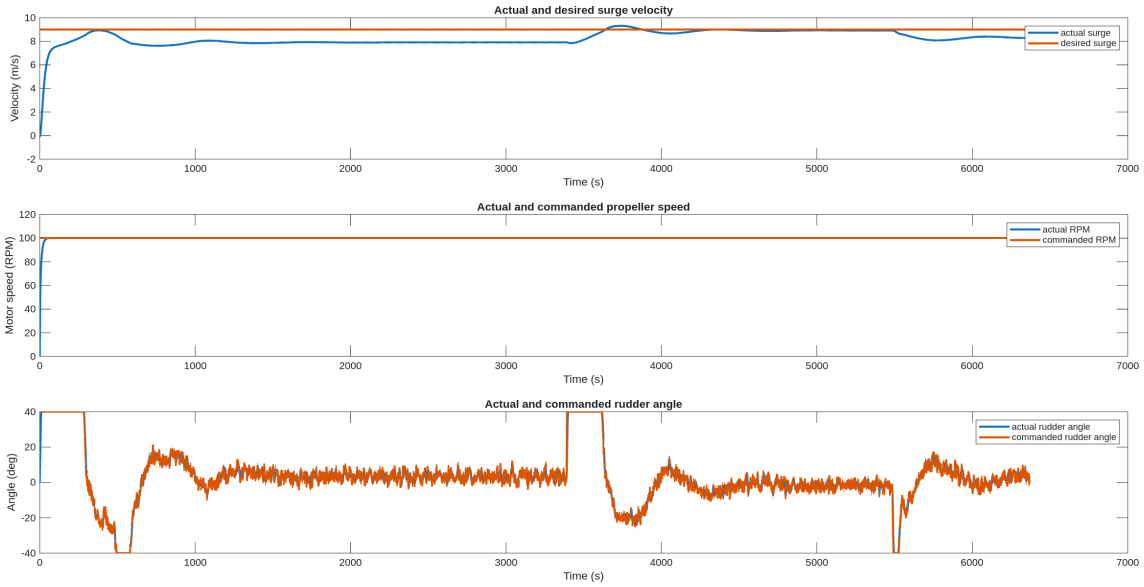

Figure 13: Commanded rudder angle versus actual rudder angle.

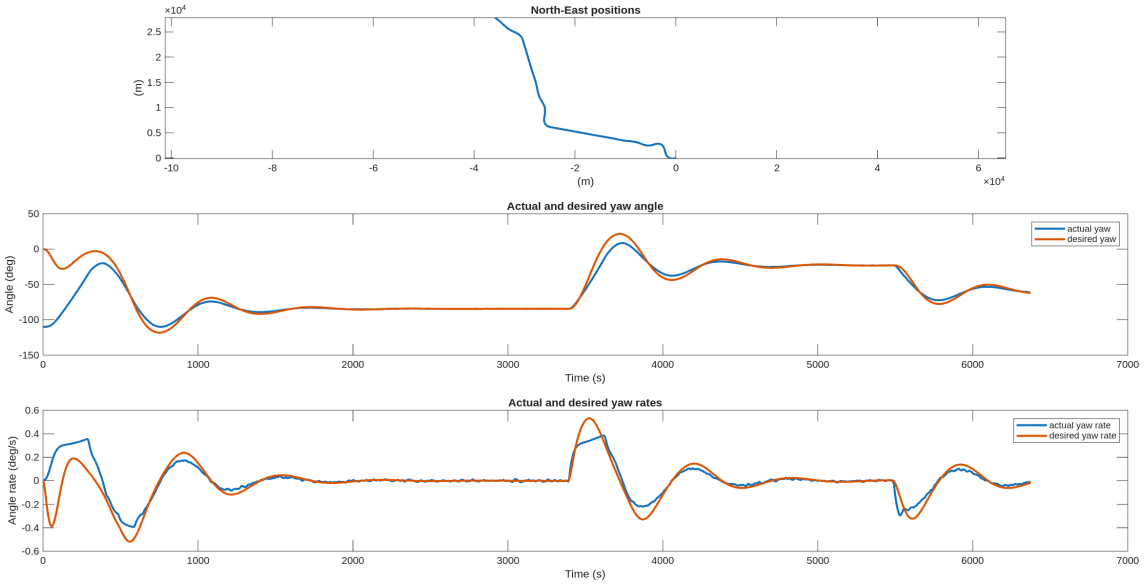

Figure 14: Desired yaw angle and yaw rate versus actual.

In this simulation, the Kalman filter was activated to estimate the yaw angle, yaw rate, and bias. The tuning parameters were set to a bandwidth frequency of $\omega_b = 0.03$ and a relative damping ratio of $\zeta = 1.8$. As seen from the figures, the overall performance is significantly improved compared to the previous case without estimation. The vessel follows the desired trajectory more smoothly, and the rudder commands are considerably less noisy.

5 Task 5

5a Wave filter

A wave filter reconstructs the low frequency vessel motion from measurements with wave-frequency oscillations. It prevents oscillations from entering the feedback loop and disturbing the control output. For large vessels, the closed-loop bandwidth is well below the wave encounter frequency, so a low-pass filter is sufficient, with the cutoff above the controller bandwidth but below the dominant wave frequencies. For small/fast vessels, where the controller bandwidth is close to the wave motion, a cascaded low-pass plus a notch filter can be used.

5b INS and model based navigation systems

The main difference between model based navigation systems and INS based systems is they way they receive information. A INS system has a IMU where it integrates the acceleration to find the rotation and speed, thus estimating the movement of the model. This method is self contained, cheap and assessable, and therefore widely adopted. The main limitation of INS is unbounded error, often called drift. This means that the longer the model is without external inputs, the bigger its uncertainty be. This can be compensated and bound the error with for example autistic positioning such as DVL or USBL.

A model based navigation system dose not utilize a IMU, but instead rely on sensor inputs? like thruster rpm and a mathematical model of the vessel. Based on this it calculated the next state of model. This model can work well if the inputs and mathematical model are accurate, but that is usually hard.

6 Task 6

6a 6a

The key differences between the Nomoto model-based Kalman Filter and Error-State Kalman filter (ESKF) approaches are:

State vector and system model

The model-based KF relies on knowing a low-order, approximate vessel model containing heading, yaw rate and rudder bias. The ESKF approach is based on measuring the small error states (position, velocity, orientation) instead of the full state, making it more robust against accumulating integration errors.

Sensor inputs and measurements

The KF approach measures heading, yaw rate and rudder bias for a 1DOF solution. The ESKF approach uses accelerometer and gyroscope data from e.g. an IMU to estimate the *errors* in position, velocity and orientation, covering 6DOFs.

Process and measurement noise

For the KF approach, a fictious white noise w is applied to $\dot{\psi}$, $\dot{r}$, $\dot{b}$ to account for uncertainty and unmodeled external effects. In the ESKF, the noise is derived from the characteristics of the sensors being used.

6b 6b

See appendix (main.m) for implementation of ESKF in MATLAB.

6c 6c

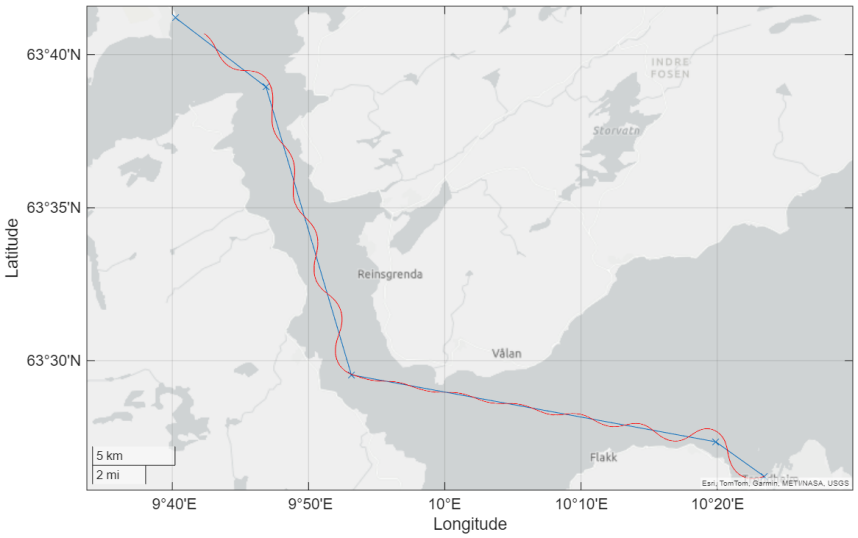

Figure 15: Path following using ESKF

Appendix

```
%% ------------------------------------------------------------------------------
% USER INPUTS
%-------------------------------------------------------------------------------
clc; clear; clear WP_selector ILOS_guidance; close all;
T_final = 6400;                % Final simulation time (s)
h = 0.1;                % Sampling time 10Hz (s)
h_gnss = 0.2;           % Sampling time 5Hz for GNSS (s)
U_ref   = 9;            % desired surge speed (m/s)

% initial states
eta_0 = [0 0 -110*pi/180]';
nu_0  = [0 0 0]';
delta_0 = 0;
n_0 = 0;
Qm_0 = 0;
% x = [ u v r x y psi delta n Qm ]'   % Ship state vector explanation
x = [nu_0' eta_0' delta_0 n_0 Qm_0]'; % The state vector can be extended with
↪  addional states here
xdot = zeros(size(x));

USE_ESKF = true; % Takes priority
USE_KF = false;  % toggle: true -> use KF estimates; false -> use noisy
↪  measurements

% 6b) ESKF
% ESKF state vector for INS
% Inc. position, velocity, accelerometer biases, orientation, and gyroscope
↪  biases.
p_ins = [0 0 0]';
v_ins = [0 0 0]';
b_acc_ins = [0 0 0]';
theta_ins = [0, 0, deg2rad(-110)]';
b_ars_ins = [0 0 0]';
x_ins = [p_ins; v_ins; b_acc_ins; theta_ins; b_ars_ins];

% Magnetic field and matitude for city #1 (Trondheim)
[m_ref, ~,mu,cityName] = magneticField(1);
% ESKF covariance matrices (from SIMaidedINSeuler)
P_prd = eye(15);
% Process noise weights: vel, acc_bias, w_nb, ars_bias
Qd = diag([0.1 0.1 0 0.001 0.001 0 0 0 0.1 0 0 0.001]);
% Measuremenet noise weights: pos, acc, compass
Rd = diag([0.1 0.1 0.1 1 1 1 0.1]);
t_slow = 0; % Used to update GNSS readings

% Reference model initialization
xd = [0; 0; 0];  % [psi_d, r_d, v_d]

% PID control initialization
e_int = 0;
%wb   = 0.06; zeta = 1.0;
wb   = 0.03; zeta = 1.8; % Tuning for task 4d

wn = wb/sqrt(1-2*zeta^2+sqrt(4*zeta^4-4*zeta^2+2));
K_nom = 7.4931e-03;
T_nom = 169.55;
```

```
%Nomoto when Uref = 9 m/s
%K_nom = 7.68e-03;
%T_nom = 174.2;
m = T_nom/K_nom;
d = 1/K_nom;
k = 0;

kp = wn^2*m-k;
kd = 2*zeta*wn*m - d;
ki = (wn/10)*kp;

% Actuator limits
delta_max  = deg2rad(40);   % max rudder angle [rad]
Ddelta_max = deg2rad(5);    % max rudder rate [rad/s]

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% MAIN LOOP
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
t = 0:h:T_final;                % Time vector
nTimeSteps = length(t);         % Number of time steps

simdata = zeros(nTimeSteps, 17); % Pre-allocate matrix for efficiency

% 3 Kalman filter initialization
rng(1)                              % reproducibility
sigma_psi = deg2rad(0.5);           % [rad] std of yaw measurement noise
sigma_r   = deg2rad(0.1);           % [rad/s] std of yaw-rate measurement noise

psi_meas_hist = zeros(nTimeSteps,1);
r_meas_hist   = zeros(nTimeSteps,1);
psi_true_hist = zeros(nTimeSteps,1);
r_true_hist   = zeros(nTimeSteps,1);
psi_hat_hist = zeros(nTimeSteps,1);
r_hat_hist   = zeros(nTimeSteps,1);
b_hat_hist   = zeros(nTimeSteps,1);

% === Continous model ===
A = [0  1     0;
     0 -1/T_nom  -K_nom/T_nom;
     0  0     0];
B = [0; K_nom/T_nom; 0];
C = [1 0 0];
E = [0 0; 1 0; 0 1];
D = 0;

% First-order discretization
Ad = eye(3) + h*A;
Bd = h*B;
Cd = C;
Ed = h*E;

% KF initialization
x_prd = [0;0;0];            % [psi_hat; r_hat; b_hat]
P_prd_KF = diag([(deg2rad(30))^2, (deg2rad(0.01))^2, (deg2rad(1))^2]);

% KF Tuning (starting point { adjust during tests)
Qd_KF = diag([1e-11, 1e-7]);    % var{w_r}, var{w_b}
Rd_KF = sigma_psi^2;           % var of psi measurement
```

```
%

% Guidance model initialization
% --- Waypoints ---
S = load('WP.mat');
WP = S.WP;

Delta_h = 600; % Lookahead distance
Rsw = 500; % Switch radius
Rstop = 100; % End radius

kappa = 1; % Design parameter for ILOS

i_end = nTimeSteps;

for i = 1:nTimeSteps
    % --- Time-varying heading reference ---
    %{
    if t(i) < 500
        psi_ref = 10 * pi/180;      % +10 deg
    else
        psi_ref = -20 * pi/180;     % -20 deg
    end
    %}

    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    % Part 2, 1a) 2D irrotational current
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    Vc = 1;
    %Vc = 0;                            % (m/s)
    betaVc = 45*pi/180;                 % 45 deg CW from N -> NE (rad)

    uc = Vc * cos(betaVc - x(6));    % BODY Surge current (m/s)
    vc = Vc * sin(betaVc - x(6));    % BODY Sway current (m/s)
    nu_c = [ uc vc 0 ]';

    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    % Part 2, 1c) Add wind here
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    Vw = 10;                % wind speed (m/s)
    beta_Vw = deg2rad(135); % wind speed direction
    rho_a = 1.247;          % air density
    c_y = 0.95;             % sway wind-force coefficient
    c_n = 0.15;             % Yaw wind-moment coefficient
    L = 161;                % Length of ship
    A_Lw = 10*L;            % Area of side view above water.

    % Tilbakemelding fra studass
    uw = Vw * cos(beta_Vw - x(6));
    vw = Vw * sin(beta_Vw - x(6));
    u_rw = x(1) - uw;
    v_rw = x(2) - vw;
    V_rw = sqrt(u_rw^2 + v_rw^2);

    gamma_rw = -atan2(v_rw,u_rw);
    Ywind = 0.5 * rho_a * V_rw^2 * c_y*sin(gamma_rw) * A_Lw;       % Equation
    ↪  from Fossen ch 10.1
    Nwind = 0.5 * rho_a * V_rw^2 * c_n*sin(2*gamma_rw) * A_Lw*L;   % Equation
    ↪  from Fossen ch 10.1
```

```
tau_wind = [0 Ywind Nwind]';

% Relative speed ocean currents
u_rc = x(1) - uc;
v_rc = x(2) - vc;
U_rc = sqrt(u_rc^2 + v_rc^2);

% Defining sidelsip and crab angle
beta_c   = atan2(x(2), max(1e-9, x(1))); % Crab angle

% Course: chi = psi + beta_c  (gjelder eksakt når Vc=0)
psi    = x(6);
chi    = ssa(psi + beta_c);

% Sideslip angle
beta = atan2(v_rc, max(1e-9, u_rc));


%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Part 3 - Task 1
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Guidance law
[xk1,yk1,xk,yk,last] = WP_selector(x(4),x(5), WP, Rsw, Rstop);
[e_y,pi_p] = crossTrackError(xk1,yk1,xk,yk,x(4),x(5));
% LOS guidance law is used to find course angle to next waypoint
% (Used to find desired heading angle for autopilot)
chi_d = LOS_guidance(e_y,pi_p, Delta_h);
%psi_ref = chi_d;            % For pure LOS without crab
psi_ref = chi_d - beta_c;    % For LOS with crab angle compensation
%psi_ref = ILOS_guidance(e_y, pi_p, kappa, Delta_h, h);
xd_dot = ref_model(xd, psi_ref);
xd = xd + h * xd_dot;
psi_d = xd(1);
r_d   = xd(2);
r_d_dot = xd_dot(2);
u_d = U_ref;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Part 2, 2d) Add the heading controller here
% Define it as a function
%
% The result should look like this:
% delta_c = PID_heading(e_psi,e_r,e_int);
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% ----- Noisy measurements (from true states) -----
psi_true = ssa(x(6));          % x = [u v r x y psi delta n Qm]'
r_true   = x(3);

psi_meas = ssa(psi_true + randn*sigma_psi );
r_meas   = r_true   + randn*sigma_r;   % not used by KF, but useful for
↪  plots

% Log for plotting
psi_meas_hist(i) = psi_meas;
r_meas_hist(i)   = r_meas;
psi_true_hist(i) = psi_true;
r_true_hist(i)   = r_true;
```

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Run Kalman Filter
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

[x_pst,P_pst,x_prd,P_prd_KF] =
↪  KF(x_prd,P_prd_KF,Ad,Bd,Ed,Cd,Qd_KF,Rd_KF,psi_meas,x(7));

% Log for plotting
% Log estimates
psi_hat_hist(i) = x_pst(1);
r_hat_hist(i)   = x_pst(2);
b_hat_hist(i)   = x_pst(3);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Part 3, 6b) ESKF
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

R_b2n = Rzyx(0, 0, x(6));

% Compute the IMU specific force vector and angular rates
g_n = [0 0 9.81]'; % NED gravity vector
% Here we assume that IMU is aligned with CO otherwise additional
↪  compensation
% (r_bmI) has to happen according to (eq. 14.13, Fossen 2021 or p. 17-18 in
↪  L9.pdf)
% Remember that x and xdot represent the motion about CO,
% the transformation is given in ship.m by:
% xg = -3.7; % CG x-ccordinate (m)
% The gyro measurement is given by the simulated angular velocity, you just
↪  need to add noise and bias


% Extract true states
w_gyro_b = [0,0,x(3)]';
f_imu_b = [xdot(1),xdot(2),0]' + Smtrx([0,0,x(3)]') * [x(1),x(2),0]' - R_b2n'
↪  * g_n; % (eq. 14.14, Fossen 2021 or p. 17-18 in L9.pdf)
gps_pos_true = [x(4:5)',0]';
gps_vel_true = R_b2n*[x(1:2)',0]';

% Sensor noise
sigma_accel = 0.001; % m/s^2 (accelerometer)
sigma_gyro = 0.0000175; % rad/s (gyroscope)
sigma_gps_pos = 0.025; % meters (RTK GPS position)
sigma_gps_vel = 0.02; % m/s (RTK GPS velocity)
sigma_compass_psi = deg2rad(0.5); % rad (compass)

w1 = sigma_accel * randn(3,1);
w2 = sigma_gyro * randn(3,1);
w3 = sigma_gps_pos * randn(3,1);
w4 = sigma_gps_vel * randn(3,1);
w5 = sigma_compass_psi * randn(1);

% Bias
b_accel = [0 0 0]';
b_gyro = [0 0 0]';

f_imu = f_imu_b + b_accel + w1;
w_imu = w_gyro_b + b_gyro + w2;
gps_pos = gps_pos_true + w3;
```

```
gps_vel = gps_vel_true + w4;
y_psi = x(6) + w5; % compass

% Positions measurements (GNSS) are slower than the sampling time
if t(i) > t_slow
    % Position aiding + compass aiding
    [x_ins,P_prd] =
    ↪  ins_euler(x_ins,P_prd,mu,h,Qd,Rd,f_imu,w_imu,y_psi,gps_pos);
    % Update the time for the next slow position measurement
    t_slow = t_slow + h_gnss;
else
    [x_ins,P_prd] = ins_euler(x_ins,P_prd,mu,h,Qd,Rd,f_imu,w_imu,y_psi);
end


% Get states from measurements, KF or ESKF
if USE_ESKF
    psi_fb = x_ins(12);             % estimated yaw from ESKF
    r_fb   = w_imu(3) - x_ins(15);  % yaw rate = gyro minus bias
elseif USE_KF
    psi_fb = x_pst(1);      % estimated yaw
    r_fb   = x_pst(2);      % estimated yaw rate
else
    %psi_fb = psi_meas;     % noisy yaw
    %r_fb = r_meas;         % noisy yaw rate
    psi_fb = x(6);          % noisy yaw
    r_fb   = x(3);          % noisy yaw rate
end

e_psi = ssa(psi_fb - psi_d);
e_r   = r_fb - r_d;
e_u   = x(1) - u_d;

% Gains from TA for testing:
%kp = 176.4330;
%Ki = 1.6448;
%Kd = 3.891e+03;

delta_unsat = -(kp*e_psi + kd*e_r + ki*e_int);

%delta_step = delta_unsat - delta_cmd;
%delta_step = max(-Ddelta_max*h, min(Ddelta_max*h, delta_step));
%delta_cmd  = delta_cmd + delta_step;

% Saturation
delta_c = min(max(delta_unsat, -delta_max), delta_max);
%delta_c = max(-delta_max, min(delta_max, delta_cmd));

% Anti-windup
e_int_dot = e_psi - (1/ki)*(delta_c - delta_unsat);
e_int = e_int + h * e_int_dot;


%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Part 2, 3e) Add open loop speed control here
% Define it as a function
%
% The result should look like this:
% n_c = open_loop_speed_control(U_ref);
```

```matlab
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    n_c = open_loop_speed_control(U_ref);
    %n_c = 10;                   % propeller speed [radians per second (rps)]

    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    % Part 2, 3f) Replace the open loop speed controller,
    % with a closed loop speed controller here
    % Define it as a function
    %
    % The result should look like this:
    % n_c = closed_loop_speed_control(u_d,e_u,e_int_u);
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    %n_c = closed_loop_speed_control(u_d, x(1), h);

    % ship dynamics
    u = [delta_c n_c]';
    [xdot,tau_total] = ship(x,u,nu_c,tau_wind);


    % store simulation data in a table (for testing)
    simdata(i,:) = [x(1:3)' x(4:6)' x(7) x(8) u(1) u(2) u_d psi_d r_d, chi,
    ↪  chi_d, beta_c, beta];

    % Euler integration
    % x = euler2(xdot,x,h);
    % Runge Kutta 4 integration
    x = rk4(@ship,h,x,u,nu_c,tau_wind);

    % --- Progress Update Logic ---
    if mod(i, floor(nTimeSteps / 10)) == 0 % Print an update every 10%
        progress = (i/nTimeSteps) * 100;
        fprintf('  %d%% complete\n', round(progress));
    elseif i == 1
        disp("  Simulation in progress:")
        fprintf('  %d%% complete\n', 0);
    end

    if last
        i_end = i;  % siste gyldige indeks i denne simuleringen

        % (valgfritt) Logg en siste rad så plott ikke blir tomt helt på slutten
        % Her logger vi med "null" kommandoer for tydelig slutt
        psi_d = x(6); r_d = 0; u_d = U_ref;   % beholder samme referanser
        simdata(i,:) = [x(1:3)' x(4:6)' x(7) x(8) u(1) u(2) u_d psi_d r_d, chi,
        ↪  chi_d, beta_c, beta];

        fprintf('  Reached final waypoint at t = %.1f s (step %d)\n', (i-1)*h,
        ↪  i);
        break
    end

end
simdata = simdata(1:i,:);
t = t(1:i);
%% %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% PLOTS
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
u           = simdata(:,1);                 % m/s
v           = simdata(:,2);                 % m/s
```

```
r           = simdata(:,3);                 % rad/s
r_deg       = (180/pi) * r;                 % deg/s
x           = simdata(:,4);                 % m
y           = simdata(:,5);                 % m
psi         = simdata(:,6);                 % rad
psi_deg     = (180/pi) * psi;               % deg
delta_deg   = (180/pi) * simdata(:,7);      % deg
n           = (30/pi) * simdata(:,8);       % rpm
delta_c_deg = (180/pi) * simdata(:,9);      % deg
n_c         = (30/pi) * simdata(:,10);      % rpm
u_d         = simdata(:,11);                % m/s
psi_d       = simdata(:,12);                % rad
psi_d_deg   = (180/pi) * psi_d;             % deg
r_d         = simdata(:,13);                % rad/s
r_d_deg     = (180/pi) * r_d;               % deg/s
chi         = simdata(:,14);                % rad
chi_deg     = (180/pi)* chi;                % deg
chi_d       = simdata(:, 15);               % rad
chi_d_deg   = (180/pi)*chi_d;               % deg
beta_c      = simdata(:, 16);               % rad
beta_c_deg  = (180/pi)*beta_c;               % deg
beta        = simdata(:, 17);               % rad
beta_deg    = (180/pi)*beta;                % deg

psi_meas_hist = psi_meas_hist(1:length(t));
r_meas_hist   = r_meas_hist(1:length(t));
psi_true_hist = psi_true_hist(1:length(t));
r_true_hist   = r_true_hist(1:length(t));
psi_hat_hist = psi_hat_hist(1:length(t));
r_hat_hist   = r_hat_hist(1:length(t));
b_hat_hist   = b_hat_hist(1:length(t));

%%
figure(3)
figure(gcf)
subplot(311)
plot(y,x,'linewidth',2); axis('equal')
title('North-East positions'); xlabel('(m)'); ylabel('(m)');
subplot(312)
plot(t,psi_deg,t,psi_d_deg,'linewidth',2);
title('Actual and desired yaw angle'); xlabel('Time (s)');  ylabel('Angle
↪  (deg)');
legend('actual yaw','desired yaw')
subplot(313)
plot(t,r_deg,t,r_d_deg,'linewidth',2);
title('Actual and desired yaw rates'); xlabel('Time (s)');  ylabel('Angle rate
↪  (deg/s)');
legend('actual yaw rate','desired yaw rate')

figure(2)
figure(gcf)
subplot(311)
plot(t,u,t,u_d,'linewidth',2);
title('Actual and desired surge velocity'); xlabel('Time (s)'); ylabel('Velocity
↪  (m/s)');
legend('actual surge','desired surge')
subplot(312)
plot(t,n,t,n_c,'linewidth',2);
```

```
title('Actual and commanded propeller speed'); xlabel('Time (s)'); ylabel('Motor
↪  speed (RPM)');
legend('actual RPM','commanded RPM')
subplot(313)
plot(t,delta_deg,t,delta_c_deg,'linewidth',2);
title('Actual and commanded rudder angle'); xlabel('Time (s)'); ylabel('Angle
↪  (deg)');
legend('actual rudder angle','commanded rudder angle')

figure(4)
figure(gcf)
plot(t, chi_deg,   'LineWidth', 2); hold on;
plot(t, chi_d_deg, 'LineWidth', 2);
plot(t, psi_deg,   'LineWidth', 2);
plot(t, beta_c_deg,'--',        'LineWidth', 1.5);
plot(t, beta_deg,  '--',        'LineWidth', 1.5);
grid on; xlabel('Time (s)'); ylabel('Angle (deg)');
title('Course (), Desired Course (_d), Heading (), Crab (_c), Sideslip ()');
legend('\chi','\chi_d','\psi','\beta_c','\beta','Location','best');

figure(5); clf; figure(gcf)
subplot(2,1,1)
plot(t, rad2deg(psi_meas_hist), 'LineWidth', 1.5); hold on
plot(t, rad2deg(psi_true_hist), '--', 'LineWidth', 2)
grid on; xlabel('Time (s)'); ylabel('\psi (deg)')
title('Yaw angle: true vs noisy measurement')
legend('noisy \psi (0.5^\circ \sigma)', 'true \psi','Location','best')
subplot(2,1,2)
plot(t, rad2deg(r_meas_hist), 'LineWidth', 1.5); hold on
plot(t, rad2deg(r_true_hist),'--', 'LineWidth', 2)
grid on; xlabel('Time (s)'); ylabel('r (deg/s)')
title('Yaw rate: true vs noisy measurement')
legend('noisy r (0.1^\circ/s \sigma)','true r','Location','best')

figure(6); clf; figure(gcf)
subplot(3,1,1)
plot(t, rad2deg(psi_true_hist), 'LineWidth', 2); hold on
plot(t, rad2deg(psi_hat_hist),  '--', 'LineWidth', 1.5)
grid on; ylabel('\psi (deg)'); title('Yaw angle: true vs KF estimate')
legend('true \psi','estimated $\hat{\psi}$',
↪  'Interpreter','latex','Location','best')
subplot(3,1,2)
plot(t, rad2deg(r_true_hist), 'LineWidth', 2); hold on
plot(t, rad2deg(r_hat_hist),  '--', 'LineWidth', 1.5)
grid on; ylabel('r (deg/s)'); title('Yaw rate: true vs KF estimate')
legend('true r','estimated $\hat{r}$','Interpreter','latex', 'Location','best')
subplot(3,1,3)
plot(t, rad2deg(b_hat_hist), 'LineWidth', 2)
grid on; xlabel('Time (s)'); ylabel('b (deg)')
title('Rudder bias estimate (no true bias available)')


%% Create objects for 3-D visualization
% Since we only simulate 3-DOF we need to construct zero arrays for the
% excluded dimensions, including height, roll and pitch
z = zeros(length(x),1);
phi = zeros(length(psi),1);
theta = zeros(length(psi),1);
```

```
% create object 1: ship (ship1.mat)
new_object('flypath3d_v2/ship1.mat',[x,y,z,phi,theta,psi],...
'model','royalNavy2.mat','scale',(max(max(abs(x)),max(abs(y)))/1000),...
'edge',[0 0 0],'face',[0 0 0],'alpha',1,...
'path','on','pathcolor',[.89 .0 .27],'pathwidth',2);

% Plot trajectories
%{
flypath('flypath3d_v2/ship1.mat',...
'animate','on','step',500,...
'axis','on','axiscolor',[0 0 0],'color',[1 1 1],...
'font','Georgia','fontsize',12,...
'view',[-25 35],'window',[900 900],...
'xlim', [min(y)-0.1*max(abs(y)),max(y)+0.1*max(abs(y))],...
'ylim', [min(x)-0.1*max(abs(x)),max(x)+0.1*max(abs(x))], ...
'zlim', [-max(max(abs(x)),max(abs(y)))/100,max(max(abs(x)),max(abs(y)))/20]);
%}

% Plot the path
pathplotter(x, y);
```